Pre-launch readiness and go-live

How a client-approved site on its `netlify.app` address becomes the client's live site on their own domain — audited, keyed, paid-path-tested and cut over. If you remember one thing: **nothing is verified until you have seen it on the live domain**, and nothing that touches money is done outside Claude Fable against a test key.

SOP ID	HGM-SOP-WEB-005	EFFECTIVE	2026-09-29
VERSION	0.3	NEXT REVIEW	2027-03-29
OWNER	Web & Content Specialist (Kyle Zinger)	TRIGGER	Once per site, after client approval
APPROVED BY	Operations Manager (Gillian) — pending	DURATION	Unconfirmed — record it on the first run

01 PURPOSE AND SCOPE

Purpose

Takes a site the client has approved on its `*.netlify.app` address and makes it ready to be a business: search and answer-engine ready, accessible, fast on every page, correct at every screen size, legally complete, wired to the client's live payment, email, booking and CRM accounts, proven with a test booking, and moved onto the client's real domain without taking their email down.

In scope

SEO and GEO. Accessibility to WCAG 2.2 AA. The Lighthouse loop. BrowserStack and real-phone checks. Legal and policy pages. Filling any keys still missing. The HGM and Meta pixels. The money path. A test booking. The DNS cutover. Confirming the site is live.

Out of scope

The build, the reviews and the client's approval — WEB-004. Google Analytics 4, Search Console and Hotjar — WEB-003, from the day the site is live.

This SOP starts when the account manager tells you the client has approved the site — WEB-004 task 8.6. It is not started from Asana. It ends where WEB-003 phase 1 begins: the site answers on the client's domain and the repository is proven to be the live site.

⚠ This is a field-test draft

Every gap still open is marked **[NEEDS INPUT]** in place. Run the first launch against it, write down what actually happened at each flag, and hand the answers to the Web & Content Specialist. Do not treat a flagged task as an instruction.

ALWAYS TRUE

Five standing rules

True on every launch.

1

Money work happens in Claude Fable, on test keys

Prices, coupons, checkout, webhooks and keys: Fable, Stripe test mode, never a real charge.

2

The client's email survives the cutover

Write down every MX and TXT record before DNS moves; prove mail arrives after.

3

Nothing is verified until it is seen on the live domain

Every check on `netlify.app` is repeated on the real domain in phase 10.

4

Claude's audit is checked, not trusted

Every audit prompt is followed by a check you do yourself. Yours is the one that counts.

5

Never invent a value

Keys and IDs come from hgmportal.com, 1Password or the client — never from another client's file.

02 PREREQUISITES

1 **The client's approval**

The account manager has told you the client approved the site at WEB-004 task 8.6, and every revision is merged to `main`.

2 **A login for hgmportal.com**

The client record holds the booking platform and the keys HGM already has. **[NEEDS INPUT: Q1 — whether the registrar and DNS host are recorded there]**

3 **1Password, with the client's vault**

The site's `.env.local` entry and the client's logins. See HGM-SOP-WEB-001.

4 **VS Code with Claude Code signed in**

Every task is done in VS Code, with Claude Code and the VS Code terminal. Claude Fable for phases 6 to 8. **[NEEDS INPUT: how Claude Fable is selected in Claude Code at HGM — the model picker in the prompt box, or a setting]**

5 **The client's Netlify team**

With the HGM web team account as a Developer (WEB-004 task 2.1).

6 **The client's GoHighLevel sub-account**

The same one the forms were built in (WEB-004 task 7.4).

7 **BrowserStack, with its keys in .env.local**

So Claude can run the responsive check itself. **[NEEDS INPUT: Q22 — the HGM BrowserStack account, the variable names for its username and access key, and whether Claude reaches it through an MCP]**

8 **A real phone**

Your own, for task 4.4. An emulator is not the same thing.

9 **The Paradise Pointe site as the reference**

For SEO voice and the legal pages. **[NEEDS INPUT: Q23 — the Paradise Pointe URL, and what "the master doc" is and where it lives]**

What are you trying to do?

Pick the phase that matches your situation and jump straight to its tasks.

PHASE 1

Confirm the build is ready to be checked

Tasks 1.1–1.3 · the day the account manager says go

PHASE 2

SEO and GEO

Tasks 2.1–2.4 · before any audit reads the pages

PHASE 3

Accessibility, then push to production

Tasks 3.1–3.5 · after SEO, before performance

PHASE 4

Performance and responsive checks

Tasks 4.1–4.4 · against production

PHASE 5

Legal and policy pages

Tasks 5.1–5.3 · client wording, or HGM's

PHASE 6

Fill the missing keys and install them

Tasks 6.1–6.5 · every blank from task 1.2

PHASE 7

The money path, in Claude Fable

Tasks 7.1–7.5 · test keys only, never a real charge

PHASE 8

Test booking and the full flow

Tasks 8.1–8.6 · one booking, cancelled after

PHASE 9

DNS cutover

Tasks 9.1–9.12 · the point of no return

PHASE 10

Confirm live and hand to WEB-003

Tasks 10.1–10.4 · everything again, on the real domain

Phase 1 — Confirm the build is ready to be checked

It may be a week since anyone touched this site. Start from the merged build, with a filled environment, on a branch.

1.1 Pull main and confirm the deployed site is main

Open the repository in VS Code, fetch, fast-forward, then compare the commit Netlify has deployed with `origin/main`.

TERMINAL

```
git fetch origin && git status -sb
git pull --ff-only
git rev-parse origin/main    # compare with the commit shown on the Netlify deploy
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Fetch origin and tell me how far behind main is. If a fast-forward is possible, pull. Then ask the
Netlify connector for get-deploy-for-site on this site and tell me whether commit_ref equals origin/main.
Make no changes.
```

Expected result

`git status -sb` shows `main...origin/main` with no `[behind N]`, and the deployed commit equals `origin/main`.

1.2 Check .env.local is filled, by length

Print each key with the length of its value, never the value. If the file is missing, restore it from the client's 1Password entry (HGM-SOP-WEB-001). Every blank is a key the client still owes — write the names down for phase 6.

TERMINAL — LENGTHS ONLY

```
awk -F= '/^[A-Za-z_][A-Za-z0-9_]*=/ { print $1, length($2) }' .env.local
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Check .env.local: list every key with the LENGTH of its value. Never print a value. Tell me which keys
are blank.
```

Never print or paste a value

Lengths, not contents. A value pasted into a chat or a screenshot is a key that has to be rotated.

Expected result

You have the list of blank keys. Phase 6 fills them.

1.3 Make the pre-launch branch and open the cost row

Create a branch named `pre-launch` off `main` following HGM-SOP-WEB-002, and add today's row to `TOKENS.md`.

TERMINAL

```
git checkout -b pre-launch
```

Expected result

Phases 2 and 3 land on `pre-launch` and reach production together in task 3.5.

Phase 2 — SEO and GEO

Give every page its titles, descriptions and social shares, then make the site legible to answer engines. Check on your machine; nothing is pushed until task 3.5.

2.1 Set meta titles, descriptions and social shares for every page

In Claude Code, run the prompt below with the Paradise Pointe URL and the master doc attached. **[NEEDS INPUT: Q23 — the Paradise Pointe URL and the master doc]**

CLAUDE PROMPT — FILL IN THE TWO ANGLE-BRACKET VALUES

```
Set the meta title, meta description and social share (Open Graph and Twitter card) for every page on this site. Use <Paradise Pointe URL> as the reference for a site that is already done, and read <master doc> to understand the voice and tone to write in. List every page and the values you set.
```

Expected result

A list of every page with its title, description and share image. Compare it against the sitemap; no page may be missing.

2.2 Check the tags on three pages yourself

With the site running on `localhost`, open the home page, the stays page and one listing page. View source and find `<title>`, `<meta name="description"`, `og:title`, `og:description` and `og:image` on each.

POINT & CLICK — THEN REPEAT FOR DESCRIPTION

```
Browser: View Page Source (Cmd + Option + U in Chrome) → Cmd + F "og:image".
```

TERMINAL — USE YOUR LOCAL PORT

```
curl -s http://localhost:3000/ | grep -o '<meta property="og:[a-z:]*" content="[^"]*"'
```

Image to be added

(image to be added) — One page's rendered `<head>` showing the title, description and social tags.

Expected result

Every tag is present and the share image URL loads. A title that reads as a template placeholder is a fail.

2.3 Run the GEO audit — schema and llms.txt

In Claude Code, run the prompt below.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Do a full GEO audit of this site. Optimise the site for generative engines: add schema where it is
missing so that each page's entity is described, update existing schema so LLMs know what each page is
about, and create an llms.txt file at the site root describing the site, its pages and its properties.
Fill schema only from data the site actually has; leave out anything you would have to invent. List every
change.
```

Expected result

JSON-LD on every page type, one builder function per entity (WEB-003 task 7.2 explains why), and `llms.txt` at the site root.

2.4 Check the schema and read llms.txt

Open Google's Rich Results Test, choose **Code**, and paste the page source of the stays page from `localhost`, then one listing page. Open `http://localhost:3000/llms.txt` and read it.

> **Test the list page, not just the detail page**
>
> On Ridge & Falls the listings page emitted a thin copy of each property with the same `@id` as the full node on the property page, and the test scored every cabin "2 critical". The list page is where it shows.

Expected result

Zero critical issues on both pages, and `llms.txt` describes this client and no other.

Phase 3 — Accessibility, then push to production

Aim for WCAG 2.2 AA without breaking the design, check it by hand, then put everything from phases 2 and 3 on production.

3.1 Run the accessibility audit prompt

In Claude Code, paste the prompt exactly as written.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE, VERBATIM

```
Do a quick accessibility check as in an audit on the website and correct all the accessibility issues. Ask questions if you are unsure. Ensure the website is keyboard navigable, AA rated where possible, add alt tags to all images and anything else that needs to be done to enhance accessibility.
```

Expected result

A list of issues found and fixed, and a list of questions.

3.2 Answer every question before letting it continue

The prompt says "ask questions if you are unsure" — read each one and answer it. A typical one is what a decorative image's alt text should be, or whether a client colour that fails contrast may change.

AA is the aim, not the promise

Fix what can be fixed without changing how the site looks. If a fix would visibly break the design, skip it and list it in the pull request in task 3.5. HGM does not tell clients a site is AA certified.

Keep the client's swatches

Do not let the audit silently change a brand colour. Note the contrast ratio and tell the account manager the passing alternative, so the client decides.

Expected result

No question is left unanswered, and no brand colour has changed without the client's say.

3.3 Check keyboard navigation and alt text yourself

On `localhost`, put the mouse down. Tab from the top of the home page through the search widget, onto the stays page, into a listing and into the booking form. Then confirm no image on the stays page lacks an `alt` attribute.

POINT & CLICK

```
Browser: click in the address bar, then press Tab repeatedly.
Watch for the focus ring on every link, button and field.
Press Enter on the search button; keep tabbing into the results.
```

TERMINAL — PRINTS IMAGES WITH NO ALT

```
curl -s http://localhost:3000/stays | grep -o '<img[^>]*>' | grep -v 'alt='
```

Image to be added

(image to be added) — The site with a keyboard focus ring visible on a button, proving the indicator is not clipped. Ring the focus ring.

Expected result

You can reach and operate every control with the keyboard, focus is visible at every step, and the grep prints nothing.

3.4 Check the rest of WCAG 2.2 AA with Claude

Run the prompt below. It covers the AA criteria the first audit does not ask about by name, including the six added in WCAG 2.2.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Check this site against these WCAG 2.2 AA criteria and report pass, fail or not applicable for each, with the page and element for every fail: 1.3.1 info and relationships (headings, lists, labels in the markup); 1.4.3 text contrast and 1.4.11 non-text contrast; 1.4.4 text resizes to 200% and 1.4.10 reflow at 320px wide with no sideways scrolling; 1.4.12 text spacing; 2.4.4 link purpose; 2.4.6 headings and labels; 2.4.7 focus visible; 2.4.11 focus not obscured by the sticky header or pop-up; 2.5.7 anything done by dragging also works with a click; 2.5.8 targets at least 24 by 24 pixels; 3.1.1 page language set; 3.2.3 consistent navigation; 3.2.6 help and contact links in the same place on every page; 3.3.1 and 3.3.3 form errors named in text with a fix suggested; 3.3.7 the booking form does not ask twice for what the guest already entered; 4.1.2 name, role and value on every custom control. Fix fails that do not change how the site looks. List the rest and ask me before changing anything visual.
```

First run of this prompt

It replaces the criteria list that was missing from v0.1. Note anything it misses or gets wrong on the first launch so it can be tightened.

Expected result

A pass, fail or not-applicable for every criterion in the prompt, with fails fixed or listed for the pull request.

3.5 Push and merge to production

Commit on `pre-launch`, push, open a pull request to `main` following HGM-SOP-WEB-002, and list any accessibility fix you skipped in its description. Merge it and wait for the production build.

TERMINAL

```
git add -A && git commit -m "feat: SEO, GEO and accessibility" && git push -u origin pre-launch
```

Expected result

The `netlify.app` production address carries every change from phases 2 and 3. Phase 4 measures this build.

Phase 4 — Performance and responsive checks

Measured against production, not your machine. Two loops, each followed by a look.

4.1 Run the Lighthouse loop against production

Make a new branch off `main`, then run the prompt below with the `netlify.app` address. Merge its fixes the way task 3.5 does and re-run Lighthouse on production to confirm. **[NEEDS INPUT: Q21 — Lighthouse through the CLI or a Netlify plugin, and the accepted floor if a page cannot reach 100]**

CLAUDE PROMPT — FILL IN THE NETLIFY.APP ADDRESS

```
Next create a loop that checks Lighthouse against <netlify.app URL> and continues to loop until
everything and every page is 100/100. Make the fixes on this branch.
```

A loop is blunt here too

A Lighthouse loop is safer than a matching loop because the score is a hard number, but if it stops improving, stop it and fix the remaining items by name. A third-party embed — the GoHighLevel form, a map — can cap a score.

Image to be added

(image to be added) — A Lighthouse run on the home page showing the four scores at the end of the loop. Ring the Performance score.

Expected result

A per-page table of the four scores on production, all 100 or with the reason a score is capped.

4.2 Run Lighthouse yourself on three pages

In Chrome, open DevTools, choose the **Lighthouse** panel, and run the home page, one listing page and the stays page on production — mobile and desktop.

Screenshot 4.2 to capture

A Lighthouse run showing the four 100s. Crop to the four score circles and the page URL; ring the Performance score.

Expected result

The scores you see match the table Claude reported. If they do not, the loop measured a different build.

4.3 Run the responsive check on BrowserStack

In Claude Code, run the prompt below. **[NEEDS INPUT: Q22 — whether Claude reaches BrowserStack through an MCP or the keys alone, and the device list HGM tests against]**

CLAUDE PROMPT — FILL IN THE NETLIFY.APP ADDRESS

```
Check mobile responsiveness on BrowserStack against <netlify.app URL>, all breakpoints on the website and all device sizes including tablets should be perfect.
```

Image to be added

(image to be added) — The BrowserStack device grid part-way through a responsive pass.

Expected result

A report per breakpoint and device, phones and tablets, with any layout fault fixed and merged.

4.4 Look at it on a real phone

Open the production address on your own phone. Walk home, stays, a listing and the booking form. Open the mobile menu and the pop-up.

Expected result

Nothing overflows, nothing is cut off, the sticky search widget behaves, and the pop-up fits the screen. BrowserStack emulates; a phone is the truth.

Phase 5 — Legal and policy pages

The client's own wording where they have it, HGM's where they do not, laid out like Paradise Pointe.

5.1 Collect the wording

Ask the account manager for the client's cancellation policy, privacy policy and terms of use. If the client has none, use HGM's boilerplate pages. **[NEEDS INPUT: where HGM's boilerplate privacy policy and terms of use are kept]**

Expected result

Three pieces of text — the client's, or HGM's boilerplate for whatever the client does not have.

5.2 Build the pages to match Paradise Pointe

Make a branch off `main`, attach the wording, and run the prompt below. Paradise Pointe is the reference for the design of these pages as well as their placement.

CLAUDE PROMPT — FILL IN THE PARADISE POINTE URL

```
Add a privacy policy page and a terms of use page, matching the layout, design and placement of the same pages on <Paradise Pointe URL>. Use the attached wording verbatim. If the terms of use are a PDF, link to it rather than retyping it. Add both to the footer.
```

Expected result

Both pages are reachable from the footer on every page, look like the rest of the site, and any PDF opens.

5.3 Make the cancellation policy identical everywhere

The policy appears on every listing page, on the checkout and confirmation pages and in the confirmation email. Ask for one source of truth, then merge phase 5 the way task 3.5 does.

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
The cancellation policy must be identical on every listing page, the checkout and confirmation pages, and in every email the site sends. Move it to one constant and render it from there everywhere. Then list every place it appears.
```

TERMINAL EXPECT ONE FILE

```
grep -rl "<a distinctive phrase from the policy>" app lib components
```

Expected result

One definition. A grep for a distinctive phrase from the policy finds it in that one file only.

Phase 6 — Fill the missing keys and install them

Every blank from task 1.2 gets a value, goes onto Netlify, and goes back to 1Password. Fable from task 6.3.

6.1 Get every key that is still blank

For each blank from task 1.2, follow the WEB-004 task that says where it comes from: Stripe test keys 4.6, Resend 4.7, Meta 4.8, GoHighLevel 4.9, the booking platform 4.10. Ask the account manager to chase anything the client still owes.

Test keys only, on this SOP

A key beginning `sk_live_` does not go into `.env.local` on anyone's machine. If the client sends live keys, put them in 1Password and stop; the live swap is task 10.3.

Expected result

Re-running task 1.2's length check shows no blank key that phase 7 needs.

6.2 Put the new values on Netlify

Set the new variables on Netlify the way WEB-004 task 4.11 does: one named variable at a time, never a bulk import.

Image to be added

(image to be added) — Netlify Environment variables with the full list for this site, values redacted.

Expected result

Every variable in `ENV.template` exists on Netlify with a non-empty value.

6.3 Register the webhooks and confirm the keys, in Claude Fable

Switch to Claude Fable and run the prompt below. **[NEEDS INPUT: Q14 — what the "Supabase access token" is: a personal access token, or the service_role key WEB-004 already collected] [NEEDS INPUT: Q15 — the webhook endpoint paths the template exposes, where the Stripe signing secret is set, and whether webhooks are re-registered on the live domain in task 9.6]**

CLAUDE PROMPT — FABLE — PASTE INTO CLAUDE CODE WITH FABLE SELECTED

```
The Stripe publishable and secret test keys have been added. Register the webhooks for both the booking platform and Stripe. The Supabase access token has been added. The Resend API key has been added. Confirm the checkout page is functional, the webhooks are correctly set up, and the Resend, Meta and GHL keys are working. Add everything to the project on Netlify.
```

"Working" means a request went through

Claude can only confirm a key by making a call with it. A Resend key is confirmed by a test email in Resend's log, a GHL key by a contact appearing in the sub-account, a Meta key by a test event. "The key looks valid" is not a check.

Screenshot 6.8 to capture

The Stripe webhook endpoint dialog in test mode. Crop to the endpoint URL and the events list; ring the add-endpoint button.

Image to be added

(image to be added) — The Stripe webhook endpoint dialog with the event list selected. Ring the Add endpoint control.

Expected result

Stripe's test-mode webhook list shows the site's endpoint; the booking platform shows its webhook; a report names each key as working.

6.4 Add the HGM and Meta pixels to the site

[NEEDS INPUT: Q19 — what the HGM pixel is, and whether the template's env-gated Meta pixel block covers both, so that this is a variable rather than code]

TERMINAL — PRINTS THE PIXEL ID THE PAGE CARRIES

```
curl -s https://<site-name>.netlify.app/ | grep -o "fbq('init', '[0-9]*')"
```

Image to be added

(image to be added) — Meta Events Manager showing test traffic arriving from the site. Ring the live event row.

Expected result

The pixel script is in the live HTML of the production home page, with the client's ID.

6.5 Write everything back to 1Password

Save the current `.env.local` into the client's vault following HGM-SOP-WEB-001.

Expected result

1Password holds every key phase 7 needs, and a blank only where the client still owes a value phase 7 does not need.

Phase 7 — The money path, in Claude Fable

Everything here is in Claude Fable, in Stripe test mode. A mistake here charges a real guest the wrong amount once the site is live, so this phase is never merged with another and never rushed.

7.1 Confirm you are in Claude Fable and in test mode

Check the model shown in Claude Code is Claude Fable, and check that every Stripe value in `.env.local` begins with a test prefix.

TERMINAL — PRINTS THE FIRST EIGHT CHARACTERS ONLY

```
awk -F= '/STRIPE/ { print $1, substr($2,1,8) }' .env.local
```

Expected result

The model is Fable and every Stripe value starts `pk_test_` or `sk_test_`. A live prefix stops this phase.

7.2 Walk the checkout page and the payment intent

Ask Fable to trace the money path end to end and report, without changing anything.

CLAUDE PROMPT — FABLE — PASTE INTO CLAUDE CODE WITH FABLE SELECTED

```
Trace the money path from the property page to a confirmed booking: where the price is calculated, where deposits or fees are added, where the coupon is applied, where the Stripe payment intent is created and with what amount and currency, and what the webhook does on payment success. Report it as a numbered list with the file and function for each step. Make no changes.
```

Expected result

A list you can follow in the code. The amount sent to Stripe is the amount shown to the guest, in the currency the client charges in.

7.3 Set up the coupon

[NEEDS INPUT: promo-code campaign links document — how the 100% coupon and any campaign coupons are created, where they live, and how they are applied]

Expected result

The 100% coupon applies at checkout and the total reads zero.

7.4 Build the campaign links

[NEEDS INPUT: promo-code campaign links document — the link format, which pages accept a code from the URL, and how a code is validated]

Expected result

A campaign link opens the site with its code applied, and an invalid code is refused with a message.

7.5 Prove it in Stripe test mode

Make a test payment with a Stripe test card, then open Stripe in test mode and find the payment and the webhook delivery.

Never a real charge

If at any point a live key, a real card or a live-mode dashboard is in front of you, stop and go back to task 7.1.

Image to be added

(image to be added) — Stripe test mode showing a succeeded payment intent for the test booking, with the discounted total. Ring the amount.

Expected result

The payment appears with the right amount, and the webhook endpoint from task 6.3 shows a delivered event for it. A payment with no webhook delivery means the booking was paid for and never recorded.

Phase 8 — Test booking and the full flow

One booking, as a guest would make it, from the search widget to the confirmation email — then cleaned up.

8.1 Walk the booking flow as a guest, with dates and data

Start at the home page search widget. Choose dates, guests and pets if the site asks; check the stays page shows them; open a listing; start the booking. At every step confirm the dates, guest count and price carry through unchanged.

Expected result

What you chose on the home page is what the checkout shows.

8.2 Make the test booking

Complete the booking using the 100% coupon from WEB-004 task 4.6. **[NEEDS INPUT: Q25 — a real property and real dates, or a designated test property]**

Expected result

The confirmation page, with the booking reference.

8.3 Check the confirmation email in Resend

Open Resend and find the confirmation email in its log. Open the message and read it: the cancellation policy from task 5.3, the dates, the property, the amount.

Screenshot 8.3 to capture

The Resend email log with the confirmation delivered. Crop to the row; ring the delivered status.

Image to be added

(image to be added) — The Resend log showing the confirmation email delivered, recipient redacted. Ring the delivered status.

Expected result

The email was delivered, not just sent, and its content matches the confirmation page.

8.4 Check the reservation reached the booking platform, then cancel it

Open the client's booking platform account and find the reservation. Cancel it. **[NEEDS INPUT: Q25 — whether the test booking creates a real reservation and who cancels it]**

Expected result

The reservation existed, and the dates are free again.

8.5 Check the downstream systems

Confirm the guest appeared as a contact in the client's GoHighLevel sub-account, and that the Meta event fired. **[NEEDS INPUT: how HGM checks a Meta test event — Events Manager test events, or not checked at this stage]**

Expected result

One new contact, one purchase event.

8.6 Do the manual check, then merge

Open every page in the sitemap on desktop and look at each one. Merge anything still on a branch through a pull request per HGM-SOP-WEB-002, and confirm the production build succeeds.

Expected result

The `netlify.app` production address carries every change from phases 2 to 8. This is what goes live in phase 9.

Phase 9 — DNS cutover

The point of no return. Do it with the final checklist open, with a second person watching the first time, and with the client's email records written down before anything changes.

9.1 Agree the cutover moment

Agree with the account manager the day and time of the cutover and who on the client side will be reachable during it, then post it in the **client websites** space. **[NEEDS INPUT: Q7 — who decides the moment and what window HGM offers]**

POST IN THE CLIENT WEBSITES SPACE

<client> cutover booked for <day> at <time>. Client contact on the day: <name>. Pre-launch checks are complete.

Expected result

The day, the time and the client contact are posted. Without all three, the cutover does not start.

9.2 Find where the domain lives

Open the client's record in hgmportal.com and read the registrar (where the domain is owned) and the DNS host (where its records are served). If they are not recorded, look the nameservers up. **[NEEDS INPUT: Q1 and Q2 — whether the registrar and DNS host are on the portal record, and whether Cloudflare is common]**

TERMINAL — REPLACE <CLIENT-DOMAIN>

```
dig +short NS <client-domain>
dig +short A <client-domain>
dig +short MX <client-domain>
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Run dig for NS, A, MX and TXT on <client-domain>. Tell me who hosts the DNS, where the site currently points, and whether the domain carries email. Make no changes.
```

Image to be added

(image to be added) — The registrar or Cloudflare account page showing which nameservers the domain is using. Ring the nameserver list.

Expected result

You know the registrar, the DNS host, and whether the domain carries email.

9.3 Write down every existing record

In the DNS host's record editor, export or screenshot every record — A, AAAA, CNAME, MX, TXT, SRV — and save the capture to the client's 1Password vault with today's date.

Screenshot 10.2 to capture

The DNS host's record editor showing the full record list before any change. Crop to the table; ring the MX rows.

The MX and TXT records are the client's email

MX routes the client's mail; TXT carries SPF, DKIM and DMARC, which keep it out of spam. Whatever method task 9.8 uses, those records must exist unchanged afterwards. Moving the nameservers to Netlify DNS removes them unless they are recreated first.

Image to be added

(image to be added) — The full DNS record list before any change — this is the one you will need if you have to roll back.

Expected result

A complete record of the domain as it was, so the rollback in task 9.12 and the email check in task 9.11 have something to compare against.

9.4 Get DNS access, or a person who has it

[NEEDS INPUT: Q3 — whether HGM gets a login to the DNS host, or the client or their IT makes the change while HGM watches]

Expected result

Either you can edit the records yourself, or a named person is booked for the cutover window.

9.5 Lower the TTL

The TTL is how long, in seconds, the internet caches a record; a low one makes the change take effect fast. **[NEEDS INPUT: Q6 — whether the TTL is lowered in advance, to what, and how long before]**

Expected result

The A and CNAME records show the lowered TTL, and the old TTL has had time to expire before task 9.8.

9.6 Add the custom domain in Netlify and switch the site's own address

In the Netlify project, open **Domain management** and add `<client-domain>` and `www.<client-domain>`. Set which one is primary. Then change `NEXT_PUBLIC_SITE_URL` on Netlify to the real domain and re-register the Stripe and booking platform webhooks against it. **[NEEDS INPUT: Q10 — apex or www as primary] [NEEDS INPUT: Q28 — whether the site URL changes before or after the records change]**

Screenshot 10.5 to capture

Netlify Domain management with both domains added and the awaiting-DNS state. Crop to the domain list; ring the add-domain button.

Image to be added

(image to be added) — Netlify Domain management after the custom domain is added, showing the records Netlify asks for. Ring the record values.

Expected result

Netlify shows both domains awaiting DNS and tells you the records it wants.

9.7 Add the redirect map for old URLs

[NEEDS INPUT: Q9 — whether a redirect map exists, who writes it, and whether it is a _redirects file or netlify.toml in the template]

TERMINAL — EXPECT A 301 AND A LOCATION HEADER

```
curl -sI https://<site-name>.netlify.app/<old-path> | head -3
```

Expected result

Every URL from the old site's sitemap that has no page on the new site redirects to the right one. Test three.

9.8 Change the records at the agreed moment

[NEEDS INPUT: Q4 — the method: an A or ALIAS record for the bare domain and a CNAME for www pointing at Netlify, or moving the nameservers to Netlify DNS. The task text is written once the method is chosen.] Post in the client websites space that the cutover has started.

POST IN THE CLIENT WEBSITES SPACE

<client> cutover started at <time>. Old records saved to 1Password. Will confirm when live.

Expected result

The records are saved in the DNS host and the change is announced.

9.9 Watch it resolve

Repeat until the answers change. Propagation takes from minutes up to the old TTL.

TERMINAL — REPEAT EVERY FEW MINUTES

```
dig +short A <client-domain>
dig +short CNAME www.<client-domain>
curl -sI https://<client-domain>/ | head -1
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Every 60 seconds for up to 30 minutes, run dig +short A <client-domain> and curl -sI https://<client-domain>/. Tell me the moment the A record changes and the moment curl returns 200.
```

Image to be added

(image to be added) — A DNS lookup showing the new record resolving, from a machine outside the office network.

Expected result

The A record answers with Netlify's address and `curl` returns `HTTP/2 200`.

9.10 Confirm the certificate

In Netlify **Domain management**, check the HTTPS section shows a certificate issued for both domains. **[NEEDS INPUT: Q10 — the typical wait and whether it has ever needed a manual renew]**

Screenshot 10.9 to capture

Netlify's certificate status after cutover. Crop to the HTTPS panel; ring the issued state.

Image to be added

(image to be added) — Netlify's domain panel showing the certificate issued for the apex and www. Ring the certificate status.

Expected result

`https://<client-domain>` loads with no browser warning.

9.11 Prove the client's email survived

Send a message from your HGM address to the client's address on the domain and ask the client contact from task 9.1 to confirm they received it. Check the MX and TXT records match the capture from task 9.3.

TERMINAL — COMPARE WITH THE TASK 9.3 CAPTURE

```
dig +short MX <client-domain>
dig +short TXT <client-domain>
```

Image to be added

(image to be added) — An MX lookup for the domain after the cutover, showing the client's mail records unchanged.

Expected result

The client confirms receipt and the mail records are unchanged. This is rule 2, and it is not optional.

9.12 Rollback, if the site is wrong

[NEEDS INPUT: Q8 — the rollback: restore the saved records to point at the old host, and whether the old site is kept running and for how long]

Expected result

You know, before you need it, how to put the old site back within one TTL.

Phase 10 — Confirm live and hand to WEB-003

Everything that was checked on `netlify.app` is checked again on the live domain, and the repository is proven to be the live site.

10.1 Prove the repository is the live site

This is WEB-003 task 1.1, run for the first time.

TERMINAL — REPLACE <CLIENT-DOMAIN> AND <SITE-NAME>

```
dig +short NS <client-domain>; dig +short A <client-domain>
curl -s https://<client-domain>/ > /tmp/live.html
curl -s https://<site-name>.netlify.app/ > /tmp/host.html
diff -q /tmp/live.html /tmp/host.html
git fetch origin && git rev-parse origin/main
```

CLAUDE PROMPT — PASTE INTO CLAUDE CODE

```
Prove the repo is the live site: run dig for NS and A on <client-domain>; fetch https://<client-domain>/
and the netlify.app address and compare the HTML; ask the Netlify connector for get-deploy-for-site and
compare commit_ref with origin/main after a fetch. Make no changes. Tell me clearly if any of the three
do not match.
```

Expected result

`diff` is silent and the deployed commit equals `origin/main`.

10.2 Repeat the checks that depend on the address

On `https://<client-domain>`: the Open Graph tags (task 2.2) show the real domain, `llms.txt` loads, the Stripe test-mode webhook shows a delivered event from a checkout started on the live domain, and the pixel is in the live HTML.

TERMINAL

```
curl -s https://<client-domain>/ | grep -o '<meta property="og:url" content="[^"]*"'
curl -sI https://<client-domain>/llms.txt | head -1
curl -s https://<client-domain>/ | grep -c "fbq('init'"
```

Expected result

Every one passes on the real domain, not on `netlify.app`.

10.3 Switch Stripe to live

[NEEDS INPUT: Q11 — who swaps the test keys for live keys, where, and whether HGM ever sees the live secret key] [NEEDS INPUT: Q12 — who removes the 100% test coupon]

Expected result

The site takes real payments and the test coupon is gone.

10.4 Close out and hand to WEB-003

Add the session's row to `TOKENS.md`, save the final `.env.local` to 1Password, and post the live URL in the **client websites** space. Continue in WEB-003 the same day.

POST IN THE CLIENT WEBSITES SPACE

<client> is live at https://<client-domain> as of <time>. Email confirmed arriving. Post-launch connections (WEB-003) start <date>.

Expected result

Somebody else could start WEB-003 tomorrow from the client websites space and 1Password alone.

04 EXCEPTIONS AND EDGE CASES

The account manager asks to start before the client has approved

Do not start. WEB-005 begins at WEB-004 task 8.6; the approval comes first, whatever the deadline says.

The client cannot supply Stripe test keys

Ask for sandbox details instead. If neither arrives, phases 7 and 8 cannot run; never substitute another client's keys or a Stripe account of HGM's. Raise it with the account manager and Gillian before a cutover date is agreed.

The client's domain is on Cloudflare with the proxy on

[NEEDS INPUT: Q2 — whether HGM turns the proxy off for the Netlify records or leaves it on. Ridge & Falls was on Cloudflare with no HGM login.]

The old host also serves the client's email

Then the nameservers cannot move without the mail records being recreated first. **[NEEDS INPUT: Q5 — whether this has come up before]**

The client is on Hostaway or Cloudbeds, not Guesty

[NEEDS INPUT: Q17 — the Hostaway and Cloudbeds booking engine and webhook equivalents. AwayFrames is the first Hostaway run; record them there.]

If you are stuck

Ask Claude first, with the error and the task number. For the template's webhooks or variable names: Leshan Patterson, template owner, in the **client websites** space in Google Chat. For scope, the deadline or the client: Gillian, Operations Manager, in Google Chat — through the account manager for anything the client has to do. For a failed cutover or the client's email: **[NEEDS INPUT: Q29 — person and channel]**

05 FINAL CHECKLIST BEFORE THE CUTOVER

Do not start task 9.8 until every line is true.

- [ ] The client's approval is confirmed by the account manager, and the cutover day, time and client contact are posted in the client websites space.
- [ ] Phases 2 to 8 are complete on the `netlify.app` production build, and nothing is left unmerged.
- [ ] Every audit is checked by hand, not only reported by Claude: tags (2.2), schema (2.4), keyboard and alt (3.3), Lighthouse (4.2), a real phone (4.4).
- [ ] Privacy policy and terms of use are reachable from the footer; the cancellation policy is identical on listing pages, checkout, confirmation and email.
- [ ] Every variable in `ENV.template` is set on Netlify; `.env.local` is saved to the client's 1Password vault; no key beginning `sk_live_` is on any HGM machine.
- [ ] Webhooks are registered for Stripe and the booking platform, and a delivered event was seen for the test payment.
- [ ] Test booking made, confirmation email seen delivered in Resend, reservation seen in the booking platform and cancelled, contact seen in GoHighLevel.
- [ ] Registrar and DNS host known; every existing record captured to 1Password; MX and TXT records listed.
- [ ] DNS access confirmed, or the person making the change booked for the window.
- [ ] TTL lowered and the old TTL expired.
- [ ] Both domains added in Netlify Domain management; redirect map in place and three redirects tested.
- [ ] Rollback written down: which records to restore, and the old host still running.
- [ ] A second person is watching, if this is your first cutover.

Next: post-launch connections

HGM-SOP-WEB-003 — Google Analytics 4, Search Console, Hotjar, the structured-data check and the ownership map. Starts the day the site is live, from task 10.4.

Before this: the build

HGM-SOP-WEB-004 — the site this SOP starts from, approved by the client at its task 8.6.

06 RELATED DOCUMENTS

This SOP is the last of three before launch. It starts where the client approves the build and stops where post-launch begins.

DOCUMENT	COVERS	WHEN
HGM-SOP-WEB-004 — rebuild a live client site	The build, the reviews and the client's approval	Before this document; its task 8.6 starts this one
HGM-SOP-WEB-005 — this document	Audits, keys, money path, test booking, DNS cutover	Between the client's approval and the live domain
HGM-SOP-WEB-003 — post-launch connections	GA4, Search Console, Hotjar, structured data, the ownership map	The day the site is live, from task 10.4
HGM-SOP-WEB-002 — Asana to live	Branches, pull requests, deploy previews, merging	Referenced from tasks 1.3, 3.5 and 8.6
HGM-SOP-WEB-001 — env files in 1Password	Storing and updating the client's `.env` file	Referenced from tasks 1.2 and 6.5
The promo-code campaign links document	Coupons and campaign links in the money path	**[NEEDS INPUT: to be folded into tasks 7.3 and 7.4]**

The site is not launched when DNS resolves

It is launched when the booking flow, the payment, the email and the client's own inbox have all been seen working on the real domain. Task 10.2 is the launch; task 9.8 is just the switch.

07 REVISION HISTORY

DATE	VERSION	AUTHOR	CHANGE
2026-09-29	0.1	Kyle Zinger	Field-test draft from the notebook's Launch Process pages and the kickoff prompt. Interview answers, the WCAG audit prompt and the promo-code document outstanding; first run is AwayFrames.
2026-10-06	0.2	Kyle Zinger	Nineteen image placeholders added, weighted to the keys and cutover phases, ahead of the portal upload.
2026-10-09	0.3	Kyle Zinger	Loom review with AnhTuan. Starts at client approval, not Asana; client sign-off moved to WEB-004. Roles, definitions, troubleshooting and the platform choice removed. Push to production after accessibility; Lighthouse against production. Key directions moved to WEB-004. Privacy pop-up removed.

REMEMBER

"Nothing is verified until you have seen it on the live domain, and nothing that touches money is done outside Fable against a test key."

Next

HGM-SOP-WEB-003 — post-launch connections